You may discover more about the notification recipient or the notification itself by accessing the event's notifiable, notification, and channel attributes from inside an event listener:

```
public function handle(NotificationSending $event): void
{
 // $event->channel
 // $event->notifiable
 // $event->notification
}
```

When a notice is sent, the **`IlluminateNotificationsEventsNotification`** object is created. The notification system sends out the sent event. This holds the *notifiable* object as well as the notification instance. Listeners for this event can be registered in your `EventServiceProvider`:

```
use App\Listeners\LogNotification;
use Illuminate\Notifications\Events\NotificationSent;
protected $listen = [
 NotificationSent::class => [
  LogNotification::class,
 ],
];
```

To discover more about the notification recipient or the notification itself, you may use an event listener to obtain the event's notifiable, notification, channel, and response properties:

```
public function handle(NotificationSent $event): void
{
 // $event->channel
 // $event->notifiable
 // $event->notification
 // $event->response
}
```

Conclusion

In this chapter, we delved into the crucial aspects of leveraging e-mail and notifications in Laravel, highlighting its usefulness in tracking application performance and enabling effective communication with users. The chapter